



STRATA

Browse in context.

DOC 07 · IMPLEMENTATION PLAN

Implementation Plan

Phase-wise development from a native CEF proof of concept through Continuum, Moments, and a Windows-first release.

Version: 1.0

Date: September 2026

Status: Draft for team review

Owner: Engineering / Product

strata — local-first browser
confidential — internal working document

CONTENTS

01 How to Read This Plan

02 Phase-by-Phase Plan

03 Suggested MVP Scope

04 Flagship Demo Scenario

05 Dependencies & Sequencing Notes

How to Read This Plan

The roadmap is phase-gated (see SDLC §1): each phase below has one demoable goal, and no later phase should start in earnest until the current phase's goal is real, not just planned. Phase length is intentionally left as risk-driven rather than fixed-duration — Phase 0 in particular should be timeboxed but not schedule-boxed, because its outcome (does CEF embed reliably?) determines whether the rest of the plan is even the right plan.



Phase-by-Phase Plan

0

Native foundation

HIGHEST RISK

SPIKE

Goal: Display a real webpage (e.g. example.com) reliably inside the application's own native window.

- Stand up CEF, a minimal native host, and a basic window
- Create browser → load URL → display in native window → navigate → close, working end to end
- No React UI work starts until this is proven — see TRD §1

1

Strata shell

FOUNDATION

Goal: A functional basic browser: real window chrome around the Phase 0 native core.

- Wire together React + Tauri + Rust + CEF as one application
- Build toolbar, tabs, address bar, navigation controls, window controls
- No Continuum yet — this phase is purely “a browser that works”

2

Browser fundamentals

MVP

Goal: A usable everyday browser prototype — the team should be able to dogfood it for daily browsing.

- History, bookmarks, downloads
- Permission prompts, keyboard shortcuts
- Profiles (Personal/Work/Development/Guest) and private mode
- Internal dogfooding begins here, per the SDLC's feedback-loop principle

3

Continuum

CORE DIFFERENTIATOR

Goal: Strata remembers browsing context — silently, in the background.

- EventRecorder, StateCollector, SQLite schema (see Backend Schema doc), TimelineManager
- Record URL, title, tab, navigation, timestamp, scroll, zoom
- Rewind UI (↶ button + timeline panel) becomes usable, even before Moments exist

4

Moments

FLAGSHIP FEATURE

Goal: The defining feature actually works: save, freeze, restore, delete, rename.

- MomentManager + RestoreManager wired to the Continuum tables
- Save Moment, Freeze Moment, Restore Moment, Delete Moment, Rename Moment — all from UI and command palette
- Homepage “Continue” and “Moments” rows come alive

5

Advanced restoration

DEPTH

Goal: Restore an entire browsing workspace, not just a list of tabs.

- Split layout, multiple tabs, tab ordering, window dimensions restored together
- Page state restoration where technically possible (best-effort, per TRD §5)
- This phase's exit criterion is the full Demo Scenario below, working end to end

6

Polish

QUALITY

Goal: Strata feels like a native browser, not a web app pretending to be one.

- Animation pass, startup performance, memory usage
- Crash recovery, keyboard navigation, accessibility, security hardening
- Responsive UI pass across window sizes

Release

SHIP

Goal: Ship on Windows first; bring macOS and Linux up once Windows is solid.

- Windows build hardened, signed, packaged
- macOS and Linux follow sequentially, not in parallel
- See SDLC §5 for the CI/CD pipeline this phase relies on

Suggested MVP Scope

If Strata is being built by a small team or a single developer, do not attempt the full roadmap at once. The first impressive demo needs only this much:

```
STRATA
├── Chromium
├── Tabs
├── Address bar
├── Navigation
├── History
├── CONTINUUM
│   ├── Timeline
│   ├── Save Moment
│   ├── Freeze
│   └── Restore
```

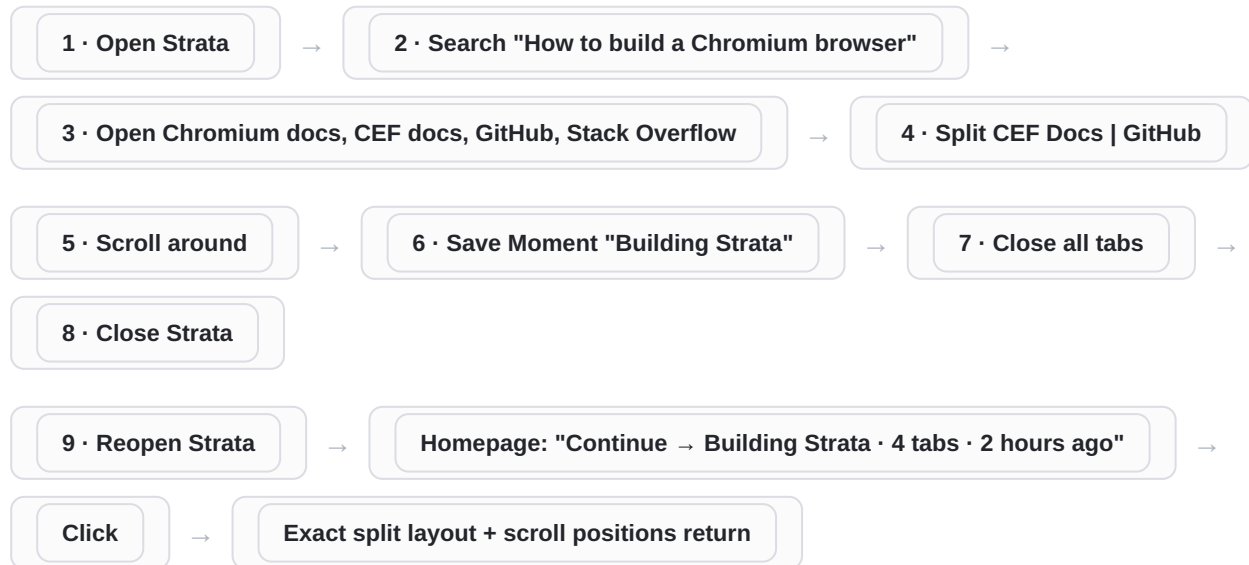
That's enough to prove the product's central idea. Everything past this (profiles, extensions, sync, themes, advanced privacy controls, workspace management, DevTools integration) is explicitly **Later** — see the feature table below.

Feature scope by tier

TIER	INCLUDES
MVP	Chromium rendering, tabs, navigation, address bar, search, back/forward/reload, keyboard shortcuts, history, bookmarks, downloads, basic permissions, private browsing, dark mode
STRATA FEATURES	Continuum, Moments, Timeline, Rewind, Freeze, Split View, context restoration
LATER	Extensions, Profiles (beyond the MVP default), Sync, Themes, advanced privacy controls, workspace management, DevTools integration

Flagship Demo Scenario

This is the acceptance test for Phase 5, and the single scenario most worth protecting schedule for. It is identical to the walkthrough in the App Flow document (§8) — repeated here as the plan's north star.



WHY THIS MATTERS

This is the moment someone understands: *"Oh. This browser is different."* Every phase before Phase 5 exists to make this scenario possible; every phase after exists to make it fast, reliable, and shippable.

Dependencies & Sequencing Notes

- Phase 3 (Continuum) cannot start meaningfully until Phase 1's navigation events are real — EventRecorder has nothing to record before the browser can navigate.
- Phase 4 (Moments) depends entirely on Phase 3's event/state tables; attempting Moments before Continuum's data model is stable means rebuilding MomentManager twice.
- Phase 5 (advanced restoration) is where Split View's `split_position` data (captured since Phase 1's window work, but unused until now) finally pays off — this is why the schema anticipates it early.
- Phase 7 (Release) platform sequencing follows the TRD's explicit guidance: Windows first, full stop, before macOS or Linux gets required CI status.

RESOURCING

Map this plan against the SDLC's role table: Phase 0–1 is native/CEF-heavy and benefits from the systems and C++ bridge roles working closely together; Phase 2–4 shifts weight toward the Continuum/data and frontend roles; Phase 6–7 is where QA/release engineering becomes the critical path.